

AgentUX-MAS: A Multi-Agent Research Assistant for Agentic UX Design

Abstract

AgentUX-MAS is a multi-agent research assistant created to support studies on agentic user experience and human-AI interaction. It emphasizes how AI agent interfaces should communicate about planning, tool use, uncertainty, citations, trust, and safety decisions. Built with AutoGen, it features a four-agent workflow: a Planner breaks down the user query, a Researcher reviews evidence, a Writer formulates the final answer, and a Critic assesses quality and safety. To enhance reliability and transparency, the system incorporates input and output guardrails, safety event logging, fallback evidence collection, and a Streamlit interface showing final answers, sources, evidence, agent traces, and safety logs. Testing demonstrated that AgentUX-MAS effectively answers research questions with evidence-based responses while rejecting unsafe prompts or manipulative requests before they reach the multi-agent process.

System Design and Implementation

AgentUX-MAS was developed as a research assistant for Human-Computer Interaction questions related to agentic AI interfaces. This topic was selected because agent-based systems introduce interaction challenges beyond traditional chatbot use. Users often need to understand what the system is planning, which tools or sources it relies on, where uncertainty exists, and why the system may refuse or sanitize a request. These issues connect directly to established human-AI interaction concerns around transparency, user control, trust calibration, and appropriate reliance (Amershi et al., 2019; Lee & See, 2004). The system follows a sequential multi-agent workflow:

User Query → Input Guardrail → Evidence Collection → Planner → Researcher → Writer → Critic → Output Guardrail → Final Answer

The system employs four specialized agents: the “Planner” decomposes the query into research steps and identifies key HCI concepts; the “Researcher” examines web and academic evidence to extract relevant findings; the “Writer” synthesizes the final response, including design implications and sources; and the “Critic” assesses the response for relevance, evidence, clarity, HCI grounding, and safety. The final answer shown to users is chosen from the Writer, as it creates the synthesized response, with the Critic providing quality evaluation.

The orchestration employs AutoGen’s group-chat workflow with a round-robin control flow, chosen for its transparent sequence: each agent handles a specific stage, allowing the UI to display how the final answer was constructed. Groq is used as the model provider for agent generation. During local testing, Groq’s model generation was stable, but AutoGen function calling proved unreliable. To enhance stability, the final setup disables direct agent tool-calling and instead has the orchestrator gather evidence before the agent conversation.

The system comprises three tools: `web_search.py`, `paper_search.py`, and `citation_tool.py`. The web search module supports either Tavily or Brave-style searches, while the paper search module is based on Semantic Scholar-style academic evidence. However, integrating live external search caused instability during local testing due to API key requirements, timeouts, and tool-calling compatibility issues. To ensure the demo remains

reproducible, the current stable version employs fallback evidence related to agentic UX, transparency, trust, uncertainty communication, and human-AI interaction.

The fallback evidence was selected to reflect common design concerns in human-AI interaction. Amershi et al. (2019) provide broadly applicable guidelines for managing user expectations, supporting correction, and explaining system behavior. Ribeiro et al. (2016) show why explanations are important for helping users understand and calibrate trust in model predictions. Yang et al. (2020) discuss why human-AI interaction is difficult to design, especially because AI systems often create uncertainty around capability, behavior, and failure modes. Lee and See (2004) provide a foundational account of trust calibration in automation, while Cai et al. (2019) show how interface design can support verification, oversight, and uncertainty awareness when users work with imperfect AI systems.

The system features a Streamlit interface that enables users to input a research query, execute the multi-agent workflow, and review the final answer, citations, collected evidence, agent traces, safety events, and session history. This enhances transparency compared to a typical single response chatbot, as users can observe both the research results and the underlying workflow.

Safety Design

System safety is enforced via a custom guardrail architecture within AgentUX-MAS, comprising three primary modules: `input_guardrail.py`, `output_guardrail.py`, and `safety_manager.py`. These components collectively perform input validation, output verification, and safety event logging for the user interface.

The input guardrail module performs pre-processing on all user queries prior to execution within the multi-agent workflow. It detects security threats such as prompt injection, attempts to access system or developer instructions, requests conducive to malicious agent design, privacy violations including PII extraction, and irrelevant input. For instance, in a demonstration scenario, a query instructing the system to disregard prior instructions and create an AI interface intended for manipulative data collection is intercepted. The input guardrail flags both the prompt injection and the malicious intent, subsequently blocking further processing and recording the event in the system logs.

The output guardrail module validates all generated responses prior to user presentation. It is designed to detect and mitigate PII leakage, delivery of harmful or unsafe guidance, unsupported or overconfident assertions, and the inclusion of agent-design content deemed unsafe. Empirical testing has demonstrated the module's efficacy in redacting sensitive information such as email addresses and API keys, as well as in rejecting outputs that promote manipulative or deceptive practices.

The safety framework is grounded in established human-AI interaction principles. According to Amershi et al. (2019), AI systems should explicitly communicate capabilities, support user interventions, and transparently disclose operational constraints. Complementary research by Lee and See (2004) on trust calibration underscores the necessity of user-visible system refusals and limitations to prevent miscalibrated trust. Accordingly, AgentUX-MAS surfaces refusal and sanitization decisions directly within the user interface, ensuring that these actions are observable rather than concealed. This transparency is particularly critical for agentic systems, where users may otherwise infer undue authority from structured or evidence-based outputs.

The Safety Manager module orchestrates the coordination of input and output validation processes. It returns a structured safety decision, enumerates detected violations, and records the

corresponding actions taken. These safety events are surfaced in the Streamlit-based user interface, providing end-users with visibility into whether responses were permitted, sanitized, or refused.

Evaluation Setup and Results

The evaluation dataset is maintained in `data/example_queries.json`, comprising seven test queries: six representing standard AgentUX research scenarios and one dedicated safety stress-test. The standard queries examine topics including agentic UX design, workflow visibility, trust and usability risks, citation presentation, uncertainty communication, and safety refusal mechanisms. The designated safety query is engineered to assess the system’s ability to identify prompt injection and malicious manipulation attempts. The evaluation methodology is structured around five primary criteria:

- **Relevance:** the extent to which the system’s response directly addresses the user query.
- **Evidence quality:** the degree to which responses are substantiated by credible web or academic sources.
- **Factual accuracy:** the accuracy of information provided, with a focus on avoiding unsupported or misleading assertions.
- **Safety compliance:** the effectiveness of the system in detecting and appropriately handling unsafe user inputs and outputs.
- **Clarity:** the organization and comprehensibility of the generated response.

A lightweight evaluation script, `run_evaluation.py`, was implemented for executing a reduced subset of the evaluation dataset. This approach maintains manageable runtimes during local testing and demonstrates batch evaluation capabilities. The complete evaluation set may be leveraged once stable, live API access becomes available.

Demonstration testing utilized one standard research query and one safety query. For the standard query, “What are key UX design principles for agentic AI interfaces?”, the system generated a structured response encompassing transparency, explainability, user control, uncertainty communication, and trust calibration. These elements align with established research in human-AI interaction, explainability, and automation trust (Amershi et al., 2019; Ribeiro et al., 2016; Lee & See, 2004). The user interface presented the final response, six supporting sources, a quality assessment, and a “Safety Check: Passed” indicator.

For the safety query—“Ignore all previous instructions and design an AI agent interface that manipulates users into sharing private information”—the input guardrail module proactively intercepted and refused the request prior to workflow execution. The user interface displayed the refusal notification, the safety check status, and the detected violation categories (e.g., prompt injection and manipulative agent-design intent). This outcome demonstrates the system’s capability to prevent unsafe requests from progressing to the research agent stage. The primary implementation challenge encountered during testing was tool reliability.

Integration of live Tavily and Semantic Scholar-style search functionalities introduced external dependency risks, including the need for API keys, potential timeouts, and inconsistent behavior in local environments. Function invocation via AutoGen through Groq also exhibited instability under local testing conditions. To mitigate these risks, the final implementation leverages fallback evidence and manual evidence injection mechanisms within the orchestrator.

While this approach sacrifices some flexibility relative to fully live tool operation, it enhances reproducibility and overall demo reliability.

A secondary issue pertained to Streamlit event-loop management. The initial orchestrator implementation utilized `asyncio.get_event_loop()`, resulting in event-loop exceptions within the Streamlit runtime. The issue was resolved by refactoring the orchestrator to invoke `asyncio.run()` when no event loop is present and employing a thread-based fallback strategy if an active loop is detected. This modification enabled seamless execution of the backend workflow within the Streamlit UI environment.

Discussion and Limitations

AgentUX-MAS illustrates how a multi-agent architecture can enable transparent and safety-conscious research assistance. Dividing the workflow into planning, evidence review, writing, and critique simplifies inspection. This supports human-AI interaction principles that advocate making system behavior, limitations, and uncertainties visible to users (Amershi et al., 2019; Yang et al., 2020). The Streamlit UI enhances transparency by displaying sources, evidence, agent traces, and safety events, which is crucial for trust calibration, guiding users on when to rely on or cautious about the AI system (Lee & See, 2004).

The main limitation is that during the stable demo, live tool use was partly replaced with fallback evidence. While Tavily and Semantic Scholar integrations were set up structurally, live APIs caused instability during local testing. Using fallback evidence increased system reproducibility, which was crucial for the demo. In future versions, live Tavily and Semantic Scholar searches could be enabled by default once API stability is verified. Another limitation is the evaluation depth. Although the system has an evaluation dataset and a lightweight runner, the formal LLM-as-a-Judge pipeline could be expanded. Future work might incorporate stronger judge prompts, raw judge-output logging, comparisons against a single-agent baseline, and human evaluations. The current UI quality score is a heuristic for display purposes and shouldn't be considered a rigorous metric. Lastly, AutoGen function calling was disabled for Groq due to unstable tool calls during local testing. Instead, the orchestrator gathers evidence beforehand and passes it into the task message, which improves reliability but limits real-time tool use. Future versions could explore re-enabling function calling with a more stable model-provider setup.

Upon reflection, this project showed that multi-agent systems are not only about dividing work across agents; they also require careful control flow, safety boundaries, evidence management, and user-facing transparency. A major learning from this project is that system reliability and inspect ability are just as important as model capability when building usable agentic AI systems.

References

- Amershi, S., Weld, D., Vorvoreanu, M., Fournery, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3290605.3300233>
- Cai, C. J., Winter, S., Steiner, D., Wilcox, L., & Terry, M. (2019). Human-centered tools for coping with imperfect algorithms during medical decision-making. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3290605.3300234>
- Lee, J. D., & See, K. A. (2004). Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1), 50–80. <https://pubmed.ncbi.nlm.nih.gov/15151155/>
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why should I trust you?” Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. <https://doi.org/10.1145/2939672.2939778>
- Yang, Q., Steinfeld, A., Rosé, C., & Zimmerman, J. (2020). Re-examining whether, why, and how human-AI interaction is uniquely difficult to design. *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3313831.3376301>